

Codedokumentation

userinterface.py

25. August 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Aufbau der Anwendung	4
2.1	Technischer Aufbau	4
2.2	Verwendete Module	4
3	Klasse HauptFenster	5
3.1	Initialisierung	5
3.2	Aufteilung des Hauptfensters	5
4	Navigation und Laden von Scans	7
4.1	Ordnerauswahl	7
4.2	Darstellung des Scans	7
5	Fingerisolation	8
5.1	Start des Isolationsvorgangs	8
5.2	Automatische Isolation	8
5.3	Manuelle Einstellung des Ellipsoids	9
6	Einzeichnen eines Untersuchungsbereichs	10
6.1	Manuelles Zeichnen	10
6.2	Automatische Farberkennung	10
7	Messfunktionen	12
7.1	Streckenmessung	12
7.2	Volumenmessungen	12
8	Farbpipette und Ereignisfilter	14
9	2D-Heatmap und 3D-Genesungsverlauf	15
9.1	2D-Heatmap	15
9.2	Vorbereitung des 3D-Genesungsverlaufs	15
9.3	Interaktive Zuordnung	16
10	Benutzerführung und Zustandsmodell	17
11	Fehlerbehandlung	18
12	Zusammenfassung	19

1 Einleitung

Der `userinterface.py` bildet die grafische Oberfläche für die Verarbeitung und Auswertung von 3D-Handscans. Die Anwendung verbindet die Darstellung der Scandaten mit mehreren Bearbeitungs- und Auswertungsfunktionen. Dazu gehören unter anderem die Isolation eines Fingers, das Einzeichnen von Untersuchungsbereichen, geometrische Messungen sowie die Darstellung eines 2D- oder 3D-Genesungsverlaufs.

Die Benutzeroberfläche ist mit `PySide6` umgesetzt. Für die 3D-Darstellung wird `PyVista` zusammen mit `pyvistaqt.QtInteractor` verwendet. Die eigentliche Verarbeitung der Scandaten ist auf mehrere Module verteilt und wird vom `userinterface.py` aus angesteuert.

Die vorliegende Dokumentation konzentriert sich auf die Klasse `HauptFenster`. Sie beschreibt insbesondere den Aufbau der Oberfläche, die Zustandsverwaltung und die wichtigsten Benutzerabläufe. Die numerischen und geometrischen Berechnungen selbst werden in den jeweils importierten Fachmodulen durchgeführt und in deren Dokumentationen genauer beschrieben und sind daher nur insoweit beschrieben, wie sie für das Verständnis der Benutzeroberfläche relevant sind.

2 Aufbau der Anwendung

2.1 Technischer Aufbau

Das Hauptfenster übernimmt vor allem die Koordination zwischen Benutzerinteraktion, 3D-Viewer und den einzelnen Verarbeitungsmodulen. Es liegen die UI-Elemente, die Zustandsverwaltung und die Aufrufe der Verarbeitungsfunktionen gemeinsam in der Klasse `HauptFenster`.

Die wesentlichen Komponenten lassen sich in vier Bereiche einteilen:

1. **Benutzeroberfläche:** PySide6-Widgets wie `QPushButton`, `QSlider`, `QTreeView` und `QLineEdit`.
2. **3D-Ansicht:** Ein `QtInteractor` dient als eingebetteter PyVista-Viewer.
3. **Zustandsverwaltung:** Attribute der Klasse speichern den aktuell geladenen Ordner, den Bearbeitungsmodus und Zwischenergebnisse.
4. **Fachmodule:** Funktionen aus Modulen wie beispielsweise `isolate_finger`, `draw_area_on_scan`, `messungen` und `heatmap3D` übernehmen die eigentliche Verarbeitung.

2.2 Verwendete Module

Die Benutzeroberfläche greift auf mehrere spezialisierte Module zurück:

Tabelle 2.1: Für die Benutzeroberfläche relevante Module

Modul	Aufgabe im UI-Kontext
<code>isolate_finger</code>	Laden und Isolation eines Fingers sowie Erzeugung und Parametrisierung des Schnitt-Ellipsoids.
<code>draw_area_on_scan</code>	Manuelles und unterstütztes Einzeichnen eines Bereichs auf dem Scan.
<code>farberkennung</code>	Erkennen farbiger Markierungen (welche vor dem Scannen auf die reale Hand gezeichnet wurden) Sowie Fehlerbehandlung, z.B. das Entfernen von Äusreisserpunkten.
<code>messungen</code>	Berechnung von Fläche, Umfang und verschiedenen Volumenwerten.
<code>heatmap2D</code>	Erzeugung der 2D-Heatmap.
<code>heatmap3D</code>	Aufbau und Speicherung des 3D-Genesungsverlaufs.
<code>farbauswahl_widget</code>	Auswahl beziehungsweise Aufnahme einer Farbe über die Pipettenfunktion.
<code>utils</code>	Unterstützung des Generator-Ablaufs bei der Fingerisolation.

Damit bleibt die Hauptklasse trotz ihrer Größe hauptsächlich für die Orchestrierung der einzelnen Arbeitsschritte zuständig. Die konkreten Berechnungen werden an die jeweiligen Fachmodule delegiert.

3 Klasse HauptFenster

3.1 Initialisierung

Die Klasse `HauptFenster` erbt von `QMainWindow`. Im Konstruktor werden zunächst Fenstereigenschaften und interne Zustände initialisiert. Das Fenster trägt den Titel “Fingerscan-Viewer” und wird mit einer Größe von 1920×1080 Pixeln angelegt.

Besonders relevant ist die Menge an Zustandsvariablen. Beispielsweise werden der aktuell geladene Ordner, der laufende Isolationsablauf, der Zeichnungsstatus und die Daten für den Genesungsverlauf gespeichert. Dadurch kann die Oberfläche abhängig vom aktuellen Arbeitsschritt unterschiedliche Bedienelemente ein- oder ausblenden.

Listing 3.1: Zentrale Zustandsvariablen der Benutzeroberfläche

```
1 self.aktueller_ordner = None
2 self.isolieren_ablauf = None
3 self.automatisch_modus_aktiv = False
4 self.zeichnungs_status = {
5     "flaeche": None,
6     "landmarken": None,
7     "punkte_eingezeichnet": None
8 }
9 self.aktuelle_teile = None
10 self.vermessen_modus_aktiv = False
11 self.ellipsoid_kontext = None
12 self.aktueller_ellipsoid_actor = None
13 self.isolieren_phase = None
14 self.genesungsverlauf_warteschlange = []
15 self.genesungsverlauf_index = 0
16 self.genesungsverlauf_gewinner = {}
17 self.genesungsverlauf_aktuell_rotiert = None
18 self._genesungsverlauf_mesh_fuer_klick = None
```

Für den 3D-Genesungsverlauf existiert zusätzlich eine Warteschlange. In ihr werden der aktuelle bereits isolierte Scan und die zugehörigen historischen Untersuchungen nacheinander verarbeitet, je nach ausgewählter Markierungsfarbe (rot, orange, gelb, grün, blau).

3.2 Aufteilung des Hauptfensters

Das zentrale Widget des Fensters verwendet ein horizontales Layout. Links befindet sich die Steuerung, rechts der 3D-Viewer. Die linke Seite besteht wiederum aus einem Bereich für die Aktionsschaltflächen und einem Dateibrowser.

Der Viewer wird mit einem `QtInteractor` realisiert:

Listing 3.2: Einbettung des PyVista-Viewers

```
1 self.plotter = QtInteractor(self.viewer_spalte)
2 self.plotter.set_background("F5F7FA")
3 viewer_layout.addWidget(self.plotter.interactor)
```

Diese Kombination ermöglicht es, den PyVista-Renderer direkt in das Qt-Fenster einzubetten. Dadurch können Qt-Interaktionen wie Buttons und Eingabefelder mit 3D-Interaktionen wie Punkt-Picking kombiniert werden.

4 Navigation und Laden von Scans

4.1 Ordnerauswahl

Scans können auf zwei Wegen geladen werden. Zum einen kann ein Scan schnell über Drag-and-Drop geladen werden. Zum anderen steht ein Dateisystem-Browser zur Verfügung.

Der Dateisystem-Browser verwendet ein `QFileSystemModel`. Der Filter beschränkt die Anzeige auf Verzeichnisse. Sobald ein ausgewähltes Verzeichnis eine OBJ-Datei enthält, wird es als Scan-Ordner übernommen.

Listing 4.1: Übernahme eines ausgewählten Scan-Ordners

```
1 def baum_klick(self, index):
2     pfad = Path(self.dateisystem_modell.filePath(index))
3     if list(pfad.glob("*.obj")):
4         self.aktueller_ordner = str(pfad)
5         self.lade_und_zeige(pfad)
```

4.2 Darstellung des Scans

Die Methode `lade_und_zeige()` sucht zunächst nach OBJ-Dateien und übergibt diese an `load_teilmeshe_mit_textur()`. Die zurückgegebenen Mesh- und Texturpaare werden im Viewer dargestellt.

Listing 4.2: Laden und Anzeigen eines Scans

```
1 obj_file = list(ordner.glob("*.obj"))
2 teile = load_teilmeshe_mit_textur(obj_file)
3
4 self.aktuelle_teile = teile
5 self.aktuelles_hand_mesh = p_v.merge(
6     [teil for teil, tex in teile]
7 )
8
9 self.plotter.clear()
10 for pv_mesh, tex in teile:
11     self.plotter.add_mesh(pv_mesh, texture=tex)
12
13 self.plotter.reset_camera()
```

Die einzelnen Mesh-Teile werden dabei nicht nur zu einem gemeinsamen Mesh zusammengeführt, sondern zusätzlich einzeln mit ihrer jeweiligen Textur in den Viewer geladen. Das ist insbesondere für die spätere farbliche Markierung relevant.

5 Fingerisolation

5.1 Start des Isolationsvorgangs

Über die Schaltfläche “Finger isolieren” wird zunächst geprüft, ob überhaupt ein Scan geladen wurde. Anschließend kann der Benutzer zwischen automatischen und manuellen Einstellungen wählen.

Listing 5.1: Start der Fingerisolation

```
1 def isolieren_klick(self):
2     if self.aktueller_ordner is None:
3         self.hinweis_label.setText("Erst einen Scan laden!")
4         return
5
6     self.zeige_basis_mesh_neu()
7     self.button_automatisch.setVisible(True)
8     self.button_manuell.setVisible(True)
9     self.isolieren_wahl_container.setVisible(True)
10    self.haupt_buttons_container.setVisible(False)
```

Die Auswahl wird dadurch umgesetzt, dass jeweils unterschiedliche Container sichtbar geschaltet werden. Die Anwendung arbeitet somit mit mehreren UI-Zuständen, ohne für jeden Zustand ein eigenes Fenster zu öffnen.

5.2 Automatische Isolation

Bei der automatischen Variante wird der Isolationsablauf als Generator erzeugt. Falls für den Scan bereits Parameter für den Ellipsoiden gespeichert wurden, werden diese geladen und an die Isolationsfunktion übergeben.

Listing 5.2: Automatischer Isolationsablauf

```
1 gespeichert = lade_isolate_finger_parameter(ordner)
2
3 if gespeichert:
4     self.isolieren_ablauf = isolate_finger(
5         str(ordner),
6         plotter=self.plotter,
7         zeige_zwischenschritte=False,
8         **gespeichert
9     )
10 else:
11     self.isolieren_ablauf = isolate_finger(
12         str(ordner),
13         plotter=self.plotter,
14         zeige_zwischenschritte=False
15     )
16
17 self.automatisch_modus_aktiv = True
18 self.button_weiter.setText("Fertig markiert")
19 self.button_weiter.setVisible(True)
```



```
20 next(self.isolieren_ablauf)
```

Der Generator ermöglicht es, den Isolationsprozess an definierten Zwischenpunkten zu unterbrechen und anschließend über die Benutzeroberfläche fortzusetzen.

5.3 Manuelle Einstellung des Ellipsoids

Bei der manuellen Variante wird der Ablauf zunächst bis zur Auswahlphase ausgeführt. Anschließend können drei Parameter über Slider angepasst werden: Breite, Länge und Unterschreitung.

Bei jeder Änderung eines Sliders wird `aktualisiere_ellipsoid_vorschau()` aufgerufen. Die Methode erzeugt einen neuen Ellipsoiden und stellt dieses halbtransparent im 3D-Viewer dar.

Listing 5.3: Aktualisierung der Ellipsoid-Vorschau

```
1 radius_faktor = self.slider_radius.value() / 100
2 laengen_faktor = self.slider_laenge.value() / 100
3 unterschreitung = self.slider_unterschreitung.value() / 100
4
5 ellipsoid = erstelle_schnitt_ellipsoid(
6     self.ellipsoid_kontext["avg_point_of_hurt_finger"],
7     self.ellipsoid_kontext["normale"],
8     self.ellipsoid_kontext["verwendete_vertices"],
9     self.ellipsoid_kontext["tiefster_punkt"],
10    radius_faktor=radius_faktor,
11    laengen_faktor=laengen_faktor,
12    unterschreitung=unterschreitung
13 )
14
15 self.aktueller_ellipsoid_actor = self.plotter.add_mesh(
16     ellipsoid,
17     color="darkcyan",
18     opacity=0.3
19 )
```

Nach der Bestätigung werden die Parameter gespeichert und an den laufenden Generator zurückgegeben. Der daraus erzeugte isolierte Scan wird anschließend erneut geladen und dargestellt.

6 Einzeichnen eines Untersuchungsbereichs

6.1 Manuelles Zeichnen

Der Zeichenmodus wird über “Bereich einzeichnen” gestartet. Die Methode `draw_main()` übernimmt die eigentliche Interaktion mit dem 3D-Scan. Der aktuelle Zustand des Zeichenvorgangs wird in `zeichnungs_status` gespeichert.

Nach Abschluss des Zeichnens werden Fläche und Umfang berechnet:

Listing 6.1: Auswertung der gezeichneten Fläche

```
1 flaecheninhalt, umfang = berechne_flaeche_und_umfang(  
2     self.zeichnungs_status["flaeche"],  
3     self.zeichnungs_status["punkte_eingezeichnet"]  
4 )  
5  
6 self.hinweis_label.setText(  
7     f"Fläche: {flaecheninhalt:.1f} mm | "  
8     f"Umfang: {umfang:.1f} mm"  
9 )
```

Anschließend wird die Untersuchungsfarbe ausgewählt. Die Fläche und die zugehörigen Landmarken werden über `save_drawn_area()` gespeichert.

6.2 Automatische Farberkennung

Neben dem manuellen Zeichnen existiert eine automatische Variante. Diese ist aktuell noch unfertig, ich möchte diese aber anpassen sobald ich mehr Testdaten zur Hand habe. Hier wird zunächst die im Farbauswahl-Widget eingestellte Farbe verwendet, um markierte Punkte im Scan zu suchen.

Der Ablauf besteht aus mehreren Schritten:

1. Erkennen der markierten Punkte anhand des Farbwerts.
2. Entfernen von Ausreißern.
3. Erzeugen eines geschlossenen Pfades.
4. Erzeugen einer Maske auf dem Hand-Mesh.
5. Schließen beziehungsweise Nachbearbeiten der Maske.
6. Extrahieren der zugehörigen Mesh-Flächen.

Listing 6.2: Automatische Erzeugung des markierten Bereichs

```
1 markierte_punkte = finde_markierungs_punkte(  
2     self.aktuelle_teile,  
3     hex_code=self.einzeichnen_farbwahl.farbe,  
4     toleranz=100.0  
5 )  
6
```

```

7 markierte_punkte = entferne_ausreisser_punkte(markierte_punkte)
8 pfad = baue_geschlossenen_pfad(markierte_punkte)
9
10 mask = get_hand_region(self.aktuelles_hand_mesh, pfad)
11 mask = schliesse_maske(
12     self.aktuelles_hand_mesh,
13     mask,
14     schritte=2
15 )
16
17 flaeche = extract_faces_of_hand(
18     self.aktuelles_hand_mesh,
19     mask
20 )

```

Dadurch kann eine bereits auf dem Scan vorhandene farbliche Markierung genutzt werden, ohne dass der Benutzer den Bereich erneut im Viewer nachzeichnen muss.

7 Messfunktionen

Die Messfunktionen sind über einen eigenen Auswahlbereich erreichbar. Grundsätzlich werden drei Arten von Messungen angeboten: Flächen- beziehungsweise Streckenmessungen, Volumenmessungen und die darauf aufbauende Darstellung von Schnittflächen.

7.1 Streckenmessung

Für eine Strecke müssen zwei Punkte auf der Oberfläche ausgewählt werden. Das Punkt-Picking wird über PyVista aktiviert. Sobald zwei Punkte ausgewählt wurden, werden die jeweils nächstgelegenen Mesh-Vertices bestimmt.

Die Strecke wird anschließend entlang der Mesh-Oberfläche als geodätische Distanz berechnet:

Listing 7.1: Berechnung einer Strecke auf der Mesh-Oberfläche

```
1 punkte_indices.append(  
2     hand_mesh.find_closest_point(punkt)  
3 )  
4  
5 if len(punkte_indices) == 2:  
6     self.plotter.disable_picking()  
7     pfad = hand_mesh.geodesic(  
8         punkte_indices[0],  
9         punkte_indices[1]  
10    )  
11     distanz = np.linalg.norm(  
12         np.diff(pfad.points, axis=0),  
13         axis=1  
14    ).sum()
```

Die Verwendung der geodätischen Strecke berücksichtigt damit die Oberfläche des Meshes und nicht nur die direkte euklidische Verbindung zwischen den beiden Punkten.

7.2 Volumenmessungen

Für das gesamte Mesh kann das in PyVista verfügbare Volumen direkt aus dem Mesh abgefragt werden. Zusätzlich existieren drei spezialisierte Varianten:

- Volumen oberhalb einer markierten Fläche,
- Volumen oberhalb eines markierten Rings, z.B. gebastelt aus einem Gummihandschuh (praktisch für den Klinikalltag)
- Volumen ab der Höhe der Fingerzwischenfalte.

Bei den ersten beiden Varianten wird zusätzlich eine transparente Schnittebene im Viewer dargestellt. Die Position beziehungsweise Orientierung der Ebene stammt aus den jeweiligen Messfunktionen.

Listing 7.2: Darstellung einer berechneten Schnittebene

```
1  ebene = p_v.Plane(  
2      center=(mitte_x, mitte_y, schnitt_hoehe),  
3      direction=(0, 0, 1),  
4      i_size=breite,  
5      j_size=tiefe  
6  )  
7  
8  self.plotter.add_mesh(  
9      ebene,  
10     color="yellow",  
11     opacity=0.35,  
12     name="schnitt_ebene"  
13 )
```

8 Farbpipette und Ereignisfilter

Für die Farbauswahl steht eine Pipettenfunktion zur Verfügung. Nach deren Aktivierung wird der Mauszeiger verändert und ein Ereignisfilter am Qt-Interactor registriert.

Beim nächsten Mausklick wird ein Screenshot des Viewers erstellt. Die Pixelkoordinaten des Klicks werden anhand des Device-Pixel-Verhältnisses angepasst und anschließend aus dem Bild ausgelesen. Der RGB-Wert wird in einen Hexadezimalwert umgewandelt und an das entsprechende Farbauswahl-Widget übergeben.

Listing 8.1: Ermittlung der Farbe aus einem Viewer-Klick

```
1 bild_array = self.plotter.screenshot(return_img=True)
2
3 hoehe, breite = bild_array.shape[:2]
4 x = min(max(x, 0), breite - 1)
5 y = min(max(y, 0), hoehe - 1)
6
7 r, g, b = bild_array[y, x][:3]
8 hex_code = f"#{r:02X}{g:02X}{b:02X}"
9
10 self._pipette_ziel_widget.setze_farbe(hex_code)
```

Der Ereignisfilter erfüllt zusätzlich eine zweite Aufgabe: Beim Verändern der Größe des Viewer-Bereichs werden die Overlay-Schaltflächen neu positioniert.

9 2D-Heatmap und 3D-Genesungsverlauf

9.1 2D-Heatmap

Die 2D-Heatmap wird über `heatmap_main()` gestartet. Die Benutzeroberfläche prüft dabei lediglich, ob ein Scan geladen wurde, und übergibt anschließend den aktuellen Ordner an das Heatmap-Modul. Diese Funktion befindet sich ebenfalls noch in Arbeit.

Listing 9.1: Start der 2D-Heatmap

```
1 def heatmap_klick(self):
2     if self.aktueller_ordner is None:
3         self.hinweis_label.setText("Erst einen Scan laden!")
4         return
5
6     heatmap_main(str(self.aktueller_ordner))
```

9.2 Vorbereitung des 3D-Genesungsverlaufs

Der 3D-Genesungsverlauf arbeitet mit mehreren Untersuchungen desselben Patienten. Zunächst wird ausgehend vom aktuellen Scan das übergeordnete Patientenverzeichnis bestimmt. Anschließend werden markierte Scans gesucht und zusammen mit den zugehörigen isolierten Scans in eine Warteschlange aufgenommen.

Der erste Eintrag der Warteschlange beschreibt den aktuellen Scan. Weitere Einträge repräsentieren historische Untersuchungen.

Listing 9.2: Aufbau der Warteschlange für den Genesungsverlauf

```
1 self.genesungsverlauf_warteschlange = [
2     {"typ": "aktuell"}
3 ]
4
5 for obj_pfad in finde_markierte_scans(patienten_ordner):
6     isolierter_name = isolierte_scan_name_aus_markierung(
7         obj_pfad.parent.name
8     )
9     isolierter_pfad = (
10         patienten_ordner
11         / "isolierte_scans"
12         / isolierter_name
13     )
14
15     if not isolierter_pfad.is_dir():
16         continue
17
18     self.genesungsverlauf_warteschlange.append({
19         "typ": "untersuchung",
20         "isolierter_pfad": isolierter_pfad,
21         "markierungs_pfad": obj_pfad
22     })
```

9.3 Interaktive Zuordnung

Für jeden Eintrag muss der Benutzer den Fingernagel im 3D-Viewer anklicken. Die daraus bestimmte Normale wird zur Ausrichtung des Meshes verwendet. Genauer wird die Normale so ausgerichtet, dass sie parallel mit der positiven Y-Achse hat oder einen Schnittpunkt mit dieser besitzt. Zusätzlich wird ein Höhenversatz berechnet, damit die Daten auf eine gemeinsame Höhe gebracht werden.

Bei historischen Untersuchungen werden anschließend die markierten Punkte geladen, transformiert und auf die entsprechenden Vertices des aktuellen Meshes abgebildet. Für jeden Vertex wird die Farbe mit der bisher gespeicherten Farbpriorität verglichen.

Listing 9.3: Zuordnung historischer Markierungen

```
1 markierung_rotiert = (R @ markierung_punkte.T).T
2 markierung_rotiert[:, 2] += versatz
3
4 for p in markierung_rotiert:
5     vertex_index = (
6         self.genesungsverlauf_aktuell_rotiert
7         .find_closest_point(p)
8     )
9
10    bisherige_farbe = (
11        self.genesungsverlauf_gewinner.get(vertex_index)
12    )
13
14    if farb_prioritaet(farbe) > farb_prioritaet(
15        bisherige_farbe
16    ):
17        self.genesungsverlauf_gewinner[
18            vertex_index
19        ] = farbe
```

Nachdem alle Untersuchungen verarbeitet wurden, werden die gewonnenen Farbgruppen im aktuellen Mesh dargestellt und der Genesungsverlauf gespeichert. Dabei wird immer die aktuellere Farbe der Markierungen dargestellt, genauer absteigend blau, grün, gelb, orange, rot.

10 Benutzerführung und Zustandsmodell

Die Benutzeroberfläche verwendet keine explizite State-Machine-Klasse. Der aktuelle Zustand wird stattdessen über Sichtbarkeitseinstellungen der Container und mehrere boolesche beziehungsweise referenzielle Attribute gesteuert.

Das Hauptmenü bildet dabei den Ausgangspunkt. Von dort kann in die Bereiche Fingerisolation, Zeichnen, Vermessen oder ein der beiden Heatmaps gewechselt werden. Während eines Unterablaufs wird das Hauptmenü ausgeblendet und ein passender Bedienbereich eingeblendet.

Ein vereinfachtes Zustandsmodell ist:

1. **Hauptmenü**
2. **Scan geladen**
3. **Fingerisolation**
 - automatische Isolation
 - manuelle Isolation
 - Ellipsoid-Anpassung
4. **Bereich zeichnen**
 - manuell
 - automatisch über Farberkennung
5. **Vermessen**
 - Strecke
 - Fläche/Umfang
 - Volumen
6. **2D-Heatmap**
7. **3D-Genesungsverlauf**

Ein vollständiges Statechart dieser Abläufe ist im zugehörigen PlantUML-Diagramm dargestellt.

11 Fehlerbehandlung

Die Oberfläche verwendet an mehreren Stellen einfache Prüfungen, bevor eine Aktion ausgeführt wird. So wird beispielsweise vor fast allen scanbezogenen Operationen kontrolliert, ob `aktueller_ordner` gesetzt ist.

Bei Dateizugriffen und Messfunktionen werden zusätzlich Ausnahmen abgefangen. Die Fehlermeldung wird anschließend über das Hinweislabel im Viewer ausgegeben. Dadurch bleibt das Hauptfenster geöffnet, auch wenn ein einzelner Verarbeitungsschritt fehlschlägt.

Beispiel:

Listing 11.1: Fehlerbehandlung beim Laden eines Scans

```
1 try:
2     obj_file = list(ordner.glob("*.obj"))
3     teile = load_teilmeshe_mit_textur(obj_file)
4 except Exception as e:
5     self.hinweis_label.setText(
6         f"Fehler beim Laden (ui, lade_und_zeige): {e}"
7     )
8     return
```

Diese Form der Fehlerbehandlung ist bewusst direkt an der Benutzeroberfläche umgesetzt, da die Anwendung den Benutzer unmittelbar über den fehlgeschlagenen Arbeitsschritt informieren soll. Das ist mir persönlich wichtig, damit der User volle Kontrolle über die Software haben kann.

12 Zusammenfassung

Die Klasse `HauptFenster` bildet die zentrale Steuereinheit des Fingerscan-Viewers. Sie verbindet die Qt-Oberfläche mit dem PyVista-Viewer und den einzelnen Modulen für Fingerisolation, Markierung, Messung und Genesungsverlauf.

Ein wesentlicher Bestandteil der Implementierung ist die Zustandsverwaltung über sichtbare UI-Container und interne Variablen. Dadurch können mehrere Arbeitsschritte innerhalb eines einzigen Fensters durchgeführt werden. Besonders bei der Fingerisolation wird zusätzlich ein Generator verwendet, um den algorithmischen Ablauf an definierten Stellen mit Benutzerinteraktionen zu verbinden.

Die eigentlichen geometrischen Berechnungen und die Verarbeitung der Scandaten bleiben in spezialisierten Modulen gekapselt. Das Hauptfenster übernimmt damit hauptsächlich die Aufgabe, Eingaben entgegenzunehmen, die passenden Verarbeitungsschritte aufzurufen, Zwischenergebnisse zu verwalten und die Ergebnisse wieder im Viewer darzustellen.